



从零开始实现Transformer ViT UNet

写在开头： 本文主要记录了皓冉在搓代码的时候的一些理解和心得，包含但不限于皓冉寒假初学的时候的一些笔记。具体的代码已经上传GitHub，<https://github.com/Arison591/Coding-from-Zero-Series>，实现的主要是一些核心算法，没有包含数据处理和训练的过程。（叠甲hh）

Transformer

先来看基础的transformer。这里先是自己手搓了一版，后面ai矫正了，前后对比图：

Original Code Issues: Before vs After			
Area	Original issue	Why it failed	Current fix
TokenEmbedding	vocab_size and padding_idx were misspelled, and forward was missing.	Calling self.TokenEmbedding(x) needs a real module forward pass.	Use vocab_size, padding_idx, and return the token embedding in forward.
PositionalEmbedding	Only encoding[0, ...] was filled.	That only writes one position instead of every sequence position.	Fill encoding[s, 0::2] and encoding[s, 1::2], then reshape to [1, max_len, d_model].
Embedding wrapper	The wrapper mixed class names and tensor logic.	Token embedding, position embedding, scaling, and dropout were not cleanly separated.	TransformerEmbedding now owns token + position + dropout.
MultiHeadAttention	The main idea was right, but there was no divisibility check.	If d_model is not divisible by num_heads, view will fail.	Add an assert and split the head reshape into split_heads.
LayerNorm	gamma/beta were defined, but gama/bata were used.	That raises AttributeError during forward.	Use the same parameter names: self.gamma and self.beta.
Decoder layer	Decoder reused norm2 and only had two norms.	Decoder has three sublayers: self-attn, cross-attn, and ffn.	Use norm1, norm2, norm3 with matching dropouts.
Transformer assembly	super(...).init__ was misspelled, scr/src names were mixed, and single layers were used like full stacks.	Constructor args, embeddings, masks, and layer stacks did not line up.	Split into EncoderLayer/DecoderLayer, stack them in Encoder/Decoder, then assemble in Transformer.
Masks	Attention accepted masks, but the full model did not create them.	The decoder could attend to future tokens, and padding tokens could affect attention.	Add make_src_mask for padding and make_tgt_mask for padding plus future-token blocking.

Main idea: your smaller pieces were close. The missing part was interface alignment: define one layer first, stack layers next, then let the top-level Transformer handle embeddings, masks, encoder, decoder, and final vocabulary projection.

那一个模块一个模块的看：

TokenEmbedding&PositionalEmbedding

代码块

```
1 class TokenEmbedding(nn.Module):
2     def __init__(self, vocab_size, d_model, padding_idx=1):
```

```

3         super().__init__()
4         self.embedding = nn.Embedding(vocab_size, d_model,
padding_idx=padding_idx)
5         self.d_model = d_model
6
7     def forward(self, x):
8         return self.embedding(x) * math.sqrt(self.d_model)
9
10    class PositionalEmbedding(nn.Module):
11        def __init__(self, d_model, max_len, device):
12            super().__init__()
13
14            # shape: [1, max_len, d_model], 前面的 1 用来和 batch 维度广播相加
15            encoding = torch.zeros(max_len, d_model, device=device)
16            position = torch.arange(0, max_len, device=device).unsqueeze(1)
17            div_term = torch.exp(torch.arange(0, d_model, 2, device=device) * (-
math.log(10000.0) / d_model))
18
19            encoding[:, 0::2] = torch.sin(position * div_term)
20            encoding[:, 1::2] = torch.cos(position * div_term)
21            self.register_buffer("encoding", encoding.unsqueeze(0)) #这是在模型中注册
一个持久缓冲区, 表示这个变量不需要梯度更新, 但会随着模型一起保存和加载
22
23        def forward(self, x):
24            # x: [batch_size, seq_len]
25            seq_len = x.size(1)
26            return self.encoding[:, :seq_len, :]
27
28
29    # This wrapper is the clean place to combine token embedding, position
embedding, scaling, and dropout.
30    class TransformerEmbedding(nn.Module):
31        def __init__(self, vocab_size, d_model, max_len, dropout, device,
padding_idx=1):
32            super().__init__()
33            self.token_embedding = TokenEmbedding(vocab_size, d_model, padding_idx)
34            self.positional_embedding = PositionalEmbedding(d_model, max_len,
device)
35            self.dropout = nn.Dropout(dropout)
36
37        def forward(self, x):
38            token_emb = self.token_embedding(x)
39            pos_emb = self.positional_embedding(x)
40            return self.dropout(token_emb + pos_emb)
41
42
43    # 保留你原来的类名习惯, 后面如果已经写了 Embedding(...) 也还能用

```

```
44 Embedding = TransformerEmbedding
```

```
45
```

第一个embedding主要用的就是nn.Embedding(), 这里还专门除以根号下d_model, token embedding被scaled以保持数值稳定性。

后面的positionalEmbedding就比较复杂了, 这里主要是那个旋转编码的公式的实现, 即这个公式:

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{\frac{2i}{d}}} \right)$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{\frac{2i}{d}}} \right)$$

然后就是把两个嵌入相加再来个dropout。

MultiHeadAttention:

代码块

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, d_model, num_heads):
3         super().__init__()
4         assert d_model % num_heads == 0, "d_model 必须能被 num_heads 整除"
5
6         self.d_model = d_model
7         self.num_heads = num_heads
8         self.d_k = d_model // num_heads
9
10        self.W_q = nn.Linear(d_model, d_model)
11        self.W_k = nn.Linear(d_model, d_model)
12        self.W_v = nn.Linear(d_model, d_model)
```

```

13         self.W_combine = nn.Linear(d_model, d_model)
14         self.dropout = nn.Dropout(0.1)
15
16     def split_heads(self, x):
17         batch_size, seq_len, _ = x.size()
18         # [batch, seq_len, d_model] -> [batch, num_heads, seq_len, d_k]
19         return x.view(batch_size, seq_len, self.num_heads,
self.d_k).transpose(1, 2)
20
21     def forward(self, q, k, v, mask=None):
22         batch_size = q.size(0)
23
24         q = self.split_heads(self.W_q(q))
25         k = self.split_heads(self.W_k(k))
26         v = self.split_heads(self.W_v(v))
27
28         scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.d_k)
29         if mask is not None:
30             # mask 会广播到 [batch, num_heads, query_len, key_len]
31             scores = scores.masked_fill(mask == 0, -1e9)
32
33         attn = torch.softmax(scores, dim=-1)
34         context = torch.matmul(self.dropout(attn), v)
35         context = context.transpose(1, 2).contiguous().view(batch_size, -1,
self.d_model)
36         return self.W_combine(context)

```

这个感觉没什么好说的，都已经搓烂了hh，可能需要注意的是它的一个向量的形状，会有一个transpose(1, 2)把seq_len,num_heads做一个转置，后面换回来。

LayerNorm:

直接放代码块：

代码块

```

1 class LayerNorm(nn.Module):
2     def __init__(self, d_model, eps=1e-6):
3         super().__init__()
4         self.eps = eps
5         self.gamma = nn.Parameter(torch.ones(d_model))
6         self.beta = nn.Parameter(torch.zeros(d_model))
7
8     def forward(self, x):
9         mean = x.mean(dim=-1, keepdim=True)

```

```

10         var = x.var(dim=-1, keepdim=True, unbiased=False)
11         x = (x - mean) / torch.sqrt(var + self.eps)
12         return self.gamma * x + self.beta
13     #gamma和beta是干什么的？这是Layer Normalization中的可学习参数。gamma（缩放参数）用于调
    整归一化后的输出的尺度，而beta（偏移参数）用于调整归一化后的输出的偏移。这两个参数允许模型
    在进行归一化后重新调整数据的分布，从而提高模型的表达能力和性能。
14

```

Encoder&Decoder:

直接上代码：

代码块

```

1     # One EncoderLayer is only one block; the full Encoder below stacks several of
    these blocks.
2     class EncoderLayer(nn.Module):
3         def __init__(self, d_model, num_heads, d_ffn, dropout):
4             super().__init__()
5             self.self_attn = MultiHeadAttention(d_model, num_heads)
6             self.ffn = nn.Sequential(
7                 nn.Linear(d_model, d_ffn),
8                 nn.ReLU(),
9                 nn.Dropout(dropout),
10                nn.Linear(d_ffn, d_model),
11            )
12            self.norm1 = LayerNorm(d_model)
13            self.norm2 = LayerNorm(d_model)
14            self.dropout1 = nn.Dropout(dropout)
15            self.dropout2 = nn.Dropout(dropout)
16
17            def forward(self, x, src_mask=None):
18                attn_output = self.self_attn(x, x, x, src_mask)
19                x = self.norm1(x + self.dropout1(attn_output))
20                ffn_output = self.ffn(x)
21                x = self.norm2(x + self.dropout2(ffn_output))
22                return x
23
24    # DecoderLayer has three sublayers: masked self-attn, cross-attn, then feed-
    forward.
25    class DecoderLayer(nn.Module):
26        def __init__(self, d_model, num_heads, d_ffn, dropout):
27            super().__init__()
28            self.self_attn = MultiHeadAttention(d_model, num_heads)
29            self.cross_attn = MultiHeadAttention(d_model, num_heads)
30            self.ffn = nn.Sequential(

```

```

31         nn.Linear(d_model, d_ffn),
32         nn.ReLU(),
33         nn.Dropout(dropout),
34         nn.Linear(d_ffn, d_model),
35     )
36     self.norm1 = LayerNorm(d_model)
37     self.norm2 = LayerNorm(d_model)
38     self.norm3 = LayerNorm(d_model)
39     self.dropout1 = nn.Dropout(dropout)
40     self.dropout2 = nn.Dropout(dropout)
41     self.dropout3 = nn.Dropout(dropout)
42
43     def forward(self, x, enc_output, src_mask=None, tgt_mask=None):
44         self_attn_output = self.self_attn(x, x, x, tgt_mask)
45         x = self.norm1(x + self.dropout1(self_attn_output))
46
47         cross_attn_output = self.cross_attn(x, enc_output, enc_output,
src_mask)
48         # #为什么会有两个enc_output呢? 这是因为在DecoderLayer中, cross-attention机制需要
同时使用编码器的输出 (enc_output) 作为键 (key) 和值 (value), 而解码器当前的输入 (x) 作
为查询 (query)。因此, cross_attn函数的参数中会有两个enc_output, 一个用于生成键, 另一个
用于生成值。这种设计允许解码器在生成每个输出时都能参考编码器的全部信息, 从而更好地捕捉输入
序列的上下文关系。
49         x = self.norm2(x + self.dropout2(cross_attn_output))
50         x = self.norm2(x + self.dropout2(cross_attn_output))
51
52         ffn_output = self.ffn(x)
53         x = self.norm3(x + self.dropout3(ffn_output))
54         return x
55
56     class Encoder(nn.Module):
57         def __init__(self, vocab_size, d_model, num_heads, d_ffn, n_layers,
dropout, max_len, device, padding_idx=1):
58             super().__init__()
59             self.embedding = TransformerEmbedding(vocab_size, d_model, max_len,
dropout, device, padding_idx)
60             self.layers = nn.ModuleList([
61                 EncoderLayer(d_model, num_heads, d_ffn, dropout)
62                 for _ in range(n_layers)
63             ])
64
65         def forward(self, src, src_mask=None):
66             x = self.embedding(src)
67             for layer in self.layers:
68                 x = layer(x, src_mask)
69             return x
70

```

```

71 class Decoder(nn.Module):
72     def __init__(self, vocab_size, d_model, num_heads, d_ffn, n_layers,
73 dropout, max_len, device, padding_idx=1):
74         super().__init__()
75         self.embedding = TransformerEmbedding(vocab_size, d_model, max_len,
76 dropout, device, padding_idx)
77         self.layers = nn.ModuleList([
78             DecoderLayer(d_model, num_heads, d_ffn, dropout)
79             for _ in range(n_layers)
80 ])
81
82     def forward(self, tgt, enc_output, src_mask=None, tgt_mask=None):
83         x = self.embedding(tgt)
84         for layer in self.layers:
85             x = layer(x, enc_output, src_mask, tgt_mask)
86         return x

```

Transformer的组合模块:

代码块

```

1  # The top-level Transformer only assembles parts: masks, encoder, decoder, and
2  # output projection.
3  class Transformer(nn.Module):
4      def __init__(
5          self,
6          src_pad_idx,
7          trg_pad_idx,
8          enc_voc_size,
9          dec_voc_size,
10         d_model=512,
11         nhead=8,
12         ffn_hidden=2048,
13         n_layers=6,
14         drop_prob=0.1,
15         max_len=5000,
16         device="cpu",
17     ):
18         super().__init__()
19         self.device = torch.device(device)
20         self.src_pad_idx = src_pad_idx
21         self.trg_pad_idx = trg_pad_idx
22
23         self.encoder = Encoder(

```

```

23         enc_voc_size, d_model, nhead, ffn_hidden, n_layers, drop_prob,
max_len, self.device, src_pad_idx
24     )
25     self.decoder = Decoder(
26         dec_voc_size, d_model, nhead, ffn_hidden, n_layers, drop_prob,
max_len, self.device, trg_pad_idx
27     )
28     self.fc_out = nn.Linear(d_model, dec_voc_size)
29     '''这些mask的作用和具体内容长什么样子?
30     src_mask 用于 encoder 的自注意力, 遮住输入序列中的填充符位置; trg_mask 用于
decoder 的自注意力, 既遮住目标序列中的填充符位置, 也遮住未来位置 (即右侧位置)。
31     '''
32     def make_src_mask(self, src):
33         # src: [batch, src_len] -> [batch, 1, 1, src_len]
34         return (src != self.src_pad_idx).unsqueeze(1).unsqueeze(2) #src !=
self.src_pad_idx 是一个布尔张量, 表示哪些位置不是填充符; unsqueeze 用来增加维度以适应后
续计算。
35
36     def make_tgt_mask(self, tgt):
37         # padding mask: [batch, 1, 1, tgt_len]
38         tgt_pad_mask = (tgt != self.trg_pad_idx).unsqueeze(1).unsqueeze(2)
39
40         # subsequent mask: [1, 1, tgt_len, tgt_len], 遮住未来位置
41         tgt_len = tgt.size(1)
42         subsequent_mask = torch.tril(torch.ones((tgt_len, tgt_len),
device=tgt.device)).bool()
43         subsequent_mask = subsequent_mask.unsqueeze(0).unsqueeze(1)
44
45         return tgt_pad_mask & subsequent_mask
46
47     def forward(self, src_input, trg_input):
48         # src_input: [batch, src_len]
49         # trg_input: [batch, trg_len], 训练时通常传入右移后的目标序列
50         src_mask = self.make_src_mask(src_input)
51         trg_mask = self.make_tgt_mask(trg_input)
52
53         enc_output = self.encoder(src_input, src_mask)
54         dec_output = self.decoder(trg_input, enc_output, src_mask, trg_mask)
55         return self.fc_out(dec_output)
56

```

这里有一点很值得注意到是def make_src_mask这个函数, 这个是为了让模型不看padding的, 有意思在于这个padding的出现是因为一个batch里面的序列长度可能不一样, 所以要用0补齐

同理下面这个函数也有这个功能, 同时还多了一个防止偷看未来的功能

src_mask = 不看源句子的 pad

tgt_mask = 不看目标句子的 pad + 不偷看未来

ViT

由于ViT和transformer有很多相同的地方，所以这里只挑不一样的地方去说明一下。

ViT说白了就是把一张图片切成很多小块，把每个小块当成“词”，再像Transformer读句子一样去理解整张图。

PatchEmbedding:

先看代码：

代码块

```
1  class PatchEmbedding(nn.Module):
2      # Patch Embedding: 把整张图切成 patch, 并投影到统一的 embedding 维度。
3      def __init__(self, img_size=224, patch_size=16, in_chans=3, embed_dim=768):
4          super().__init__()
5          if img_size % patch_size != 0:
6              raise ValueError('img_size must be divisible by patch_size')
7
8          self.img_size = img_size
9          self.patch_size = patch_size
10         # patch 总数  $N = (H / P) * (W / P)$ 
11         self.num_patches = (img_size // patch_size) ** 2
12         # 卷积层一步同时完成“切块 + 线性投影”，其中  $\text{kernel\_size} = \text{stride} =$ 
           patch_size
13         self.proj = nn.Conv2d(
14             in_channels=in_chans,
15             out_channels=embed_dim,
16             kernel_size=patch_size,
17             stride=patch_size,
18         )
19
20     def forward(self, x):
21         # 输入形状:  $[B, C, H, W]$ 
22         x = self.proj(x) #  $[B, E, H/P, W/P]$ 
23         x = x.flatten(2) #  $[B, E, N]$ 
24         x = x.transpose(1, 2) #  $[B, N, E]$ 
25         return x
26
```

这里最关键的就是用一步卷积去完成切块 + 线性投影，因为 `kernel_size = stride = patch_size`，所以每个窗口刚好对应一个 patch，而且 patch 之间不重叠。

EncoderBlock:

这里是把embedding和attention和mlp做一个整合：

代码块

```
1
2 class EncoderBlock(nn.Module):
3     # 一个完整的 ViT Encoder Block: LN -> MSA -> Residual -> LN -> MLP ->
    Residual
4     def __init__(self, dim, num_heads, mlp_ratio=4.0, drop=0.0):
5         super().__init__()
6         self.norm1 = nn.LayerNorm(dim)
7         self.attn = MultiHeadAttention(dim, num_heads=num_heads,
proj_drop=drop)
8         self.norm2 = nn.LayerNorm(dim)
9         self.mlp = MLP(dim, mlp_ratio=mlp_ratio, drop=drop)
10
11     def forward(self, x):
12         # 第一条残差支路：全局 token 之间通过 self-attention 交互。
13         x = x + self.attn(self.norm1(x))
14         # 第二条残差支路：每个 token 各自经过 MLP 做非线性变换。
15         x = x + self.mlp(self.norm2(x))
16         return x
17
```

VisionTransformer:

具体的想法都在代码的注释里面了：

代码块

```
1 class VisionTransformer(nn.Module):
2     def __init__(
3         self,
4         img_size=224,
5         patch_size=16,
6         in_chans=3,
7         num_classes=1000,
8         embed_dim=768,
9         depth=4,
10        num_heads=8,
11        mlp_ratio=4.0,
12        drop=0.0,
13    ):
14        super().__init__()
15        # 第一步：把图像变成 patch token 序列。
```

```

16         self.patch_embed = PatchEmbedding(
17             img_size=img_size,
18             patch_size=patch_size,
19             in_chans=in_chans,
20             embed_dim=embed_dim,
21         )
22         num_patches = self.patch_embed.num_patches
23
24         # cls_token 用来汇聚整张图像的信息，最终拿它做分类。
25         self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim)) #这个torch函数为什么用zeros? 是因为在训练过程中，cls_token 会被优化器更新，所以初始值可以是零。虽然也可以用其他初始化方法，但零初始化是常见且简单的选择。
26         # 位置编码长度是 num_patches + 1, 因为还包含一个 cls_token。
27         self.pos_embed = nn.Parameter(torch.zeros(1, num_patches + 1,
28             embed_dim))
29         self.pos_drop = nn.Dropout(drop)
30
31         # 堆叠多个 Transformer encoder block。
32         self.blocks = nn.Sequential(
33             *[
34                 EncoderBlock(
35                     dim=embed_dim,
36                     num_heads=num_heads,
37                     mlp_ratio=mlp_ratio,
38                     drop=drop,
39                 )
40             for _ in range(depth)
41         ]
42         )
43         self.norm = nn.LayerNorm(embed_dim)
44         self.head = nn.Linear(embed_dim, num_classes)
45
46         nn.init.trunc_normal_(self.cls_token, std=0.02)
47         nn.init.trunc_normal_(self.pos_embed, std=0.02)
48
49     def forward(self, x):
50         x = self.patch_embed(x)
51         # 为 batch 中的每张图复制一个 cls_token。
52         cls_token = self.cls_token.expand(x.shape[0], -1, -1)
53         x = torch.cat((cls_token, x), dim=1)
54         # 加上位置编码，否则模型无法区分 patch 的空间顺序。
55         x = x + self.pos_embed
56         x = self.pos_drop(x)
57         x = self.blocks(x)
58         x = self.norm(x)
59         # 取第 0 个 token，也就是 cls_token，接分类头得到 logits。
60         logits = self.head(x[:, 0])

```

```
60         return logits
61
```

UNet

DoubleCov:

代码块

```
1  class DoubleCov(nn.Module):
2      def __init__(self, in_channels, out_channels, mid_channels = None):
3          super().__init__()
4          if not mid_channels:
5              mid_channels = out_channels
6          self.double_conv = nn.Sequential(
7              # padding=1 可以保持特征图宽高不变; 原来的 padding=-1 在 PyTorch 中是非
              # 法参数。
8              nn.Conv2d(in_channels, mid_channels, kernel_size=3, padding=1, bias=False),
9              nn.BatchNorm2d(mid_channels),
10             nn.ReLU(inplace=True),
11             nn.Conv2d(mid_channels, out_channels, kernel_size=3, padding=1, bias=False),
12             # 第二个 BatchNorm 的通道数要和上一层卷积输出 out_channels 对齐。
13             nn.BatchNorm2d(out_channels),
14             nn.ReLU(inplace=True)
15         )
16
17     def forward(self, x):
18         return self.double_conv(x)
```

这一个模块是一个双层卷积，第一个部分是增加通道数，同时缩小图片尺寸（后面用padding补回来了），再进行归一化处理。第二层是为了提取更细节的特征，不在通道数上面做改变。

Down:

代码块

```
1  class Down(nn.Module):
2      def __init__(self, in_channels, out_channels):
3          super().__init__()
4          self.max_pool_conv = nn.Sequential(
5              nn.MaxPool2d(2),
6              DoubleCov(in_channels, out_channels)
```

```

7         )
8
9     def forward(self,x):
10         return self.max_pool_convd(x)

```

这是下采样的过程，主要是最大池化和二层卷积。

Up:

代码块

```

1  class Up(nn.Module):
2      def __init__(self, in_channels, out_channels,bilinear = True):
3          super().__init__()
4          if bilinear:
5              self.up = nn.Upsample(scale_factor=2 ,
mode='bilinear',align_corners= True)
6              self.conv = DoubleCov(in_channels,out_channels,in_channels//2)
7
8          else:
9              # 这里不能加逗号, 否则 self.up 会变成 tuple, forward 时不能像模块一样调
用。
10             self.up =
nn.ConvTranspose2d(in_channels,in_channels//2,kernel_size = 2,stride= 2)
11             self.conv = DoubleCov(in_channels,out_channels)
12
13
14     def forward(self,x1,x2):
15         x1 = self.up(x1)
16
17         diffY = x2.size()[2]-x1.size()[2]
18         diffX = x2.size()[3]-x1.size()[3]
19
20         x1 = F.pad(x1,[diffX// 2 , diffX - diffX//2,
21                     diffY//2 , diffY - diffY//2])
22
23         x = torch.cat([x2,x1],dim = 1)
24
25         return self.conv(x)

```

上采样的时候和下采样差的不多，值得注意的是，这个bilinear的部分，使用双插值的时候，直接进行Upsample，所以这里向量的形状有点绕。

同时有一个concat的操作，把encoder的细节特征和decoder的语义特征合在一起。

UNet的拼装：

代码块

```
1  class UNet(nn.Module):
2      def __init__(self, n_channels, n_classes, bilinear = False):
3          super().__init__()
4          self.n_channels = n_channels
5          self.n_classes = n_classes
6          self.bilinear = bilinear
7
8          self.incov = DoubleCov(n_channels, 64)
9          self.down1 = Down(64, 128)
10         self.down2 = Down(128, 256)
11         self.down3 = Down(256, 512)
12         factor = 2 if bilinear else 1
13         self.down4 = Down(512, 1024 // factor)
14         self.up1 = (Up(1024, 512 // factor, bilinear))
15         self.up2 = (Up(512, 256 // factor, bilinear))
16         self.up3 = (Up(256, 128 // factor, bilinear))
17         self.up4 = (Up(128, 64, bilinear))
18         self.outc = (OutConv(64, n_classes))
19
20         # forward 必须和 __init__ 同级；原来缩进在 __init__ 里面，UNet 实例会没有可用的
    前向传播。
21         def forward(self, x):
22             x1 = self.incov(x)
23             x2 = self.down1(x1)
24             x3 = self.down2(x2)
25             x4 = self.down3(x3)
26             x5 = self.down4(x4)
27             x = self.up1(x5, x4)
28             x = self.up2(x, x3)
29             x = self.up3(x, x2)
30             x = self.up4(x, x1)
31             logits = self.outc(x)
32             return logits
33
```

这里面有个很有意思的模块，就是OutConv,这其实是一个根据需求调整输出的模块，我们会调整out_channels的大小，比如为2的时候，做的就是二类分割，out_channels越多就说明预测的类别数越多。